Software Design & Software Engineering

Mid-Semester Comprehensive Notes

TypeScript Edition

Topics Covered

- Software Engineering
- Object-Oriented Programming
- UML Diagrams
- Entity-Relationship Diagrams
- Design Patterns

Compiled for Mid-Semester Examination Preparation

Contents

1 Software Engineering **2**
- 1.1 Software Lifecycle . . . 2
  - 1.1.1 Phases of the Software Lifecycle . . . 2
  - 1.1.2 Common SDLC Models . . . 3
- 1.2 Quality Attributes . . . 3
  - 1.2.1 FURPS+ Model . . . 3
  - 1.2.2 Key Quality Attributes in Depth . . . 3

2 Object-Oriented Programming (OOP) **5**
- 2.1 Core Concepts . . . 5
  - 2.1.1 Classes & Objects . . . 5
  - 2.1.2 Constructors . . . 6
  - 2.1.3 Invariants . . . 7
- 2.2 OOP Mechanisms . . . 7
  - 2.2.1 Encapsulation . . . 7
  - 2.2.2 Abstraction . . . 8
  - 2.2.3 Inheritance . . . 10
  - 2.2.4 Polymorphism . . . 11
- 2.3 Object Relationships & Design Concepts . . . 11
  - 2.3.1 Composition . . . 12
  - 2.3.2 DTO / Domain Object . . . 13
  - 2.3.3 Observability . . . 14

3 Unified Modelling Language (UML) **15**
- 3.1 Structural UML Diagrams . . . 15
  - 3.1.1 Class Diagram . . . 15
  - 3.1.2 Class Relationships . . . 16
- 3.2 Behavioural UML Diagrams . . . 17
  - 3.2.1 Sequence Diagram . . . 17
  - 3.2.2 Use Case Diagram . . . 18

4 Entity Relationship Diagram (ERD) **19**
- 4.1 Components . . . 19
  - 4.1.1 Entities . . . 19
  - 4.1.2 Attributes . . . 19
- 4.2 ERD Notations . . . 19
  - 4.2.1 Chen Notation . . . 20
  - 4.2.2 Crow's Foot Notation . . . 20
- 4.3 Mapping Cardinalities . . . 21
- 4.4 Participation Constraints . . . 21

5 Design Patterns **22**
- 5.1 Singleton Design Pattern . . . 22
- 5.2 Factory Design Pattern . . . 24

Quick Revision Summary **28**

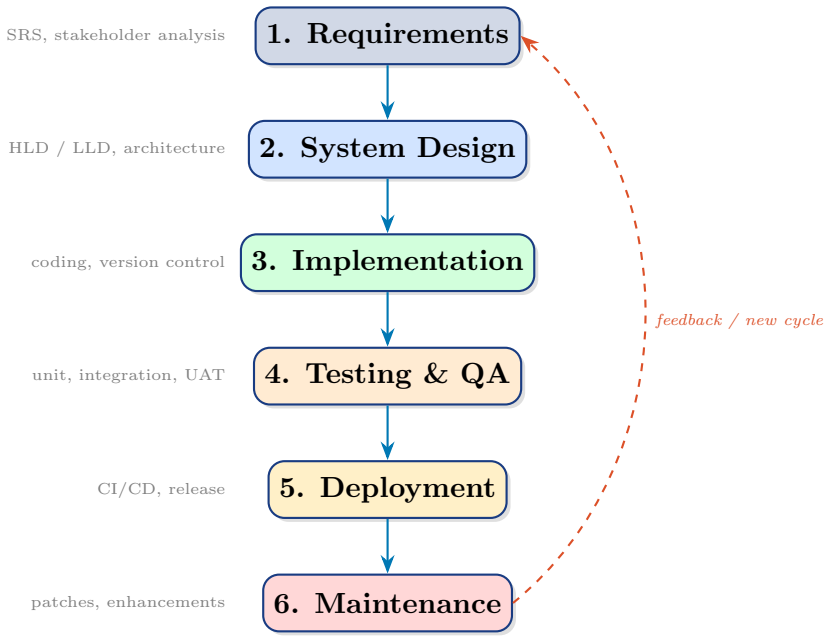
1 Software Engineering

1.1 Software Lifecycle

Definition

The **Software Development Lifecycle (SDLC)** is a structured process used by software teams to plan, design, build, test, and deliver software. It defines distinct phases, their order, and the activities performed in each phase.

1.1.1 Phases of the Software Lifecycle



Core SDLC Phases

1. **Requirements Gathering & Analysis** — Identify stakeholder needs; produce SRS; classify as *functional* vs *non-functional*.
2. **System Design** — Translate requirements into a blueprint: HLD, LLD, data models, technology choices, security protocols.
3. **Implementation** — Write source code following design documents; apply coding standards and version control.
4. **Testing & QA** — Unit, integration, system, and user acceptance testing (UAT).
5. **Deployment** — Release to production via CI/CD pipelines, blue-green or canary deployments.
6. **Maintenance** — Bug fixes, patches, performance improvements, and feature evolution. Typically the longest phase.

1.1.2 Common SDLC Models

Model	Key Characteristics
Waterfall	Sequential, phase-gated; simple but rigid.
Iterative	Repeat design-build-test cycles; improve each iteration.
Agile	Short sprints (1–4 weeks); continuous feedback; working software over documentation.
Spiral	Risk-driven; iterative with explicit risk analysis at each loop.
V-Model	Each development phase paired with a corresponding testing phase.
DevOps	Integrates development and operations; CI/CD; automated monitoring.

1.2 Quality Attributes

Definition

Quality Attributes (non-functional requirements) describe *how well* a system performs its functions, not *what* it does. They profoundly influence architectural decisions.

1.2.1 FURPS+ Model

FURPS+ Quality Attributes

Functionality
Correctness, security, compliance.

Usability Ease of learning, efficiency, error prevention, aesthetics.

Reliability Availability (uptime), MTBF, fault tolerance, recoverability.

Performance
Response time, throughput, latency, resource utilisation.

Supportability
Maintainability, testability, adaptability, scalability, portability.

+ (Plus) Design, implementation, interface, and physical constraints.

1.2.2 Key Quality Attributes in Depth

Maintainability: ease of modifying the system — measured by cyclomatic complexity, coupling/cohesion metrics. Sub-qualities: modularity, reusability, testability.

Scalability: handle growing load. *Vertical* — add more resources to one node. *Horizontal* — add more nodes.

Security: CIA Triad — Confidentiality, Integrity, Availability. Mechanisms: authentication, authorisation, encryption, auditing.

Availability:

$$\text{Availability} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Portability: transfer the system across OS, hardware, or cloud providers. **Interoperability**: communicate with external systems without special effort.

Note

Quality attributes often **trade off**: maximising security can reduce performance; high availability increases cost. Architects must balance these tensions explicitly.

2 Object-Oriented Programming (OOP)

2.1 Core Concepts

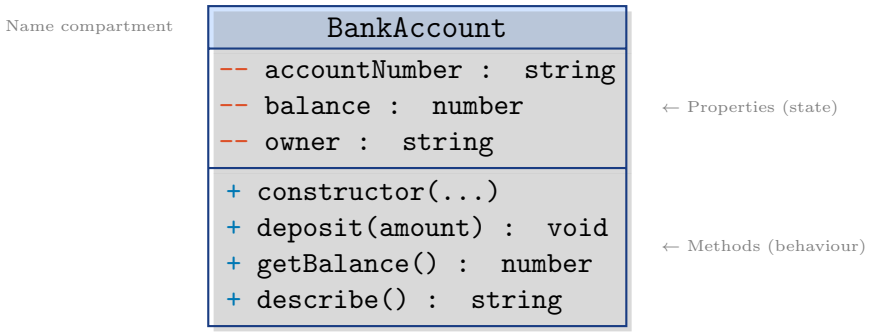
2.1.1 Classes & Objects

Definition

A **class** is a blueprint defining the structure (properties) and behaviour (methods) of objects. An **object** is a concrete instance of a class with its own specific state.

Listing 1: Class and Object in TypeScript

```typescript
// Class definition -- blueprint
class BankAccount {
  // Properties (fields)
  private accountNumber: string;
  private balance: number;
  private owner: string;

  // Constructor
  constructor(accountNumber: string, owner: string, initialBalance: number) {
    this.accountNumber = accountNumber;
    this.owner         = owner;
    this.balance       = initialBalance;
  }

  // Method (behaviour)
  deposit(amount: number): void {
    this.balance += amount;
  }

  getBalance(): number {
    return this.balance;
  }

  describe(): string {
    return `${this.owner}'s account [${this.accountNumber}]: $${this.balance}`;
  }
}

// Creating objects (instances)
const acc1 = new BankAccount("ACC001", "Alice", 5000);
const acc2 = new BankAccount("ACC002", "Bob",   1200);

console.log(acc1.describe()); // Alice's account [ACC001]: $5000
acc1.deposit(200);
console.log(acc1.getBalance()); // 5200
```



2.1.2 Constructors

Definition

A **constructor** is a special method invoked automatically when an object is created. In TypeScript it is always named `constructor`. Its purpose is to *initialise the object to a valid state.*

Listing 2: Types of Constructors in TypeScript

```typescript
class Student {
  name: string;
  age: number;
  gpa: number;

  // 1. Default / no-arg constructor
  constructor();
  // 2. Parameterised constructor
  constructor(name: string, age: number, gpa: number);
  // 3. Copy constructor (simulate via static factory)
  constructor(name?: string, age?: number, gpa?: number) {
    this.name = name ?? "Unknown";
    this.age  = age  ?? 0;
    this.gpa  = gpa  ?? 0.0;
  }

  // Copy factory (TypeScript idiom for copy constructor)
  static copy(other: Student): Student {
    return new Student(other.name, other.age, other.gpa);
  }
}

// TypeScript shorthand: declare + assign in one step
class Point {
  constructor(
    public readonly x: number,   // shorthand for this.x = x
    public readonly y: number
  ) {}
}

const p = new Point(3, 4);
console.log(p.x, p.y); // 3  4
```

2.1.3 Invariants

Definition

A **class invariant** is a condition that must always hold true for every instance — before and after every method call. Invariants define the *valid state space* of an object.

Listing 3: Enforcing Invariants in TypeScript

```typescript
class BankAccount {
  private _balance: number;

  constructor(initialBalance: number) {
    if (initialBalance < 0)
      throw new Error("Balance cannot be negative.");
    this._balance = initialBalance;   // invariant established
  }

  withdraw(amount: number): void {
    if (amount > this._balance)
      throw new Error("Insufficient funds.");
    this._balance -= amount;          // invariant maintained
  }

  // Invariant: _balance >= 0  is ALWAYS true
  get balance(): number { return this._balance; }
}
```

Note

Invariants tie into **Design by Contract**: preconditions (caller guarantees), post-conditions (method guarantees), invariants (always-true properties).

2.2 OOP Mechanisms

2.2.1 Encapsulation

Definition

Encapsulation bundles data and methods into one unit (class) while *restricting direct access* to internal state. It protects object integrity and hides implementation details.

Listing 4: Encapsulation with Getters and Setters in TypeScript

Getters and Setters

```typescript
class Person {
  private _name: string;
  private _age: number;

  constructor(name: string, age: number) {
```

```
    this._name = name;
    this.age   = age;    // route through setter for validation
  }

  // Getter
  get name(): string { return this._name; }

  // Setter with validation
  set age(value: number) {
    if (value < 0 || value > 150)
      throw new RangeError(`Invalid age: ${value}`);
    this._age = value;
  }
  get age(): number { return this._age; }
}

const alice = new Person("Alice", 30);
console.log(alice.name);  // Alice
alice.age = 31;           // OK
// alice.age = -5;        // throws RangeError
```

Note

Immutable objects (use `readonly` in TypeScript) only expose getters; state is set once in the constructor and never changes.

2.2.2 Abstraction

Definition

Abstraction hides implementation complexity and exposes only the essential features. Focus on *what* an entity does, not *how*.

Abstract Classes

Listing 5: Abstract Class in TypeScript

```
abstract class Shape {
  constructor(protected colour: string) {}

  // Abstract methods -- subclasses MUST implement
  abstract area(): number;
  abstract perimeter(): number;

  // Concrete method -- shared implementation
  printInfo(): void {
    console.log(
      `${this.constructor.name} | colour: ${this.colour}` +
      ` | area: ${this.area().toFixed(2)}`
    );
  }
}
```

```

class Circle extends Shape {
  constructor(colour: string, private radius: number) {
    super(colour);
  }
  area(): number      { return Math.PI * this.radius ** 2; }
  perimeter(): number { return 2 * Math.PI * this.radius; }
}

class Rectangle extends Shape {
  constructor(colour: string, private w: number, private h:
     number) {
    super(colour);
  }
  area(): number      { return this.w * this.h; }
  perimeter(): number { return 2 * (this.w + this.h); }
}

const c = new Circle("red", 5);
c.printInfo(); // Circle | colour: red | area: 78.54
```

Listing 6: Interfaces in TypeScript

Interfaces and Contracts

```
// Interface defines the contract
interface Printable {
  print(): void;
}

interface Serialisable {
  serialise(): string;
  deserialise(data: string): void;
}

// A class can implement multiple interfaces
class Invoice implements Printable, Serialisable {
  private content: string = "";

  print(): void          { console.log(this.content); }
  serialise(): string    { return this.content; }
  deserialise(data: string): void { this.content = data; }
}

// TypeScript-specific: interfaces for object shapes
interface User {
  readonly id: number;
  name: string;
  email?: string;   // optional property
}

// Type alias (similar to interface but for unions/intersections)
type AdminUser = User & { adminLevel: number };
```

Feature	Abstract Class	Interface
Instantiation	Cannot instantiate	Cannot instantiate
Constructor	Can have constructors	No constructors
Fields	Can have instance fields	Only type declarations
Method types	Abstract + concrete	Abstract + optional defaults
Inheritance	Single (`extends`)	Multiple (`implements`)
Use case	Shared base with partial impl.	Define a capability / role

2.2.3 Inheritance

Definition

Inheritance allows a class (*subclass*) to derive properties and behaviours from another class (*superclass*), establishing a **IS-A** relationship and enabling code reuse.

Listing 7: Inheritance in TypeScript

```typescript
class Animal {
  constructor(
    protected name: string,
    protected age: number
  ) {}

  eat(): void   { console.log(`${this.name} is eating.`); }
  describe(): string { return `${this.name} (age ${this.age})`; }
}

class Dog extends Animal {
  constructor(name: string, age: number, private breed: string) {
    super(name, age);  // call parent constructor
  }

  bark(): void { console.log(`${this.name} says: Woof!`); }

  // Method overriding
  override describe(): string {
    return `${super.describe()} | Breed: ${this.breed}`;
  }
}

const rex = new Dog("Rex", 3, "Labrador");
rex.eat();            // Rex is eating.
rex.bark();           // Rex says: Woof!
console.log(rex.describe()); // Rex (age 3) | Breed: Labrador
```

Inheritance Key Points

- **Method Overriding**: subclass provides a specific implementation (`override` keyword in TS).
- `super`: calls parent constructor or methods.
- `private`: not accessible in child; use `protected` for child access.
- `readonly`: prevents reassignment after construction.
- **Liskov Substitution Principle**: a subclass instance must work wherever the parent is expected.

2.2.4 Polymorphism

Definition

Polymorphism allows a single interface or method to work with objects of different types. The correct method is selected based on the *actual* object type at runtime.

Listing 8: Runtime Polymorphism in TypeScript

```
abstract class Shape {
  abstract area(): number;
}

class Circle extends Shape {
  constructor(private r: number) { super(); }
  area(): number { return Math.PI * this.r ** 2; }
}

class Rect extends Shape {
  constructor(private w: number, private h: number) { super(); }
  area(): number { return this.w * this.h; }
}

// Polymorphic array -- different types, same interface
const shapes: Shape[] = [new Circle(5), new Rect(4, 6)];
shapes.forEach(s => console.log(s.area())); // 78.54, 24

// -- Compile-time polymorphism via function overloading --
function add(a: number, b: number): number;
function add(a: string, b: string): string;
function add(a: any,    b: any): any { return a + b; }

console.log(add(2, 3));        // 5
console.log(add("hi", "!")); // hi!
```

2.3 Object Relationships & Design Concepts

2.3.1 Composition

Definition

Composition is a **HAS-A** relationship where an object is built from other objects. The component's lifetime is *tied* to the containing object (strong ownership). If the parent is destroyed, so are its components.

Listing 9: Composition vs Aggregation in TypeScript

```typescript
// -- Composition: Engine cannot exist without Car --------------
class Engine {
  constructor(private horsepower: number) {}
  start(): void { console.log(`Engine (${this.horsepower}hp) started.`); }
}

class Car {
  private engine: Engine;  // Car "owns" Engine

  constructor(private model: string, hp: number) {
    this.engine = new Engine(hp);  // Engine born with Car
  }

  drive(): void { this.engine.start(); }
}

// -- Aggregation: Department exists independently of University --
class Department {
  constructor(public name: string) {}
}

class University {
  constructor(
    private name: string,
    private departments: Department[]  // reference, not ownership
  ) {}
}

const cs   = new Department("Computer Science");
const math = new Department("Mathematics");
const uni  = new University("MIT", [cs, math]);
// cs and math still exist even if uni is removed
```

	Composition	Aggregation
Relationship	Strong HAS-A	Weak HAS-A
Lifetime	Component tied to parent	Component independent
Dependency	Parent creates component	Parent holds reference
UML symbol	Filled diamond ♦	Open diamond ◊

2.3.2 DTO / Domain Object

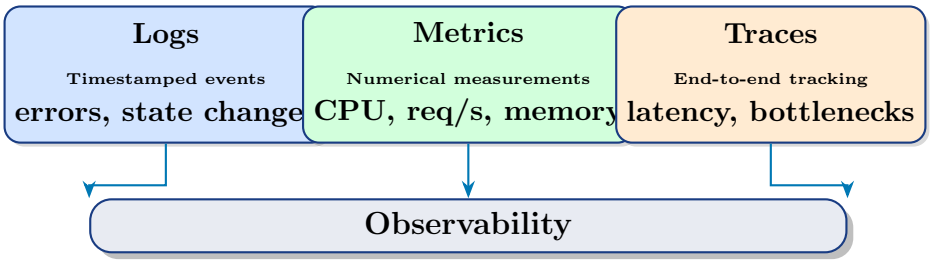
Listing 10: DTO vs Domain Object in TypeScript

```typescript
// -- Domain Object -- rich with business logic ------------------
class Order {
  private lines: OrderLine[] = [];
  private status: "pending" | "shipped" | "cancelled" = "pending";

  addItem(product: Product, qty: number): void {
    // business rules...
    this.lines.push(new OrderLine(product, qty));
  }

  totalPrice(): number {
    return this.lines.reduce((sum, l) => sum + l.subtotal(), 0);
  }

  cancel(): void {
    if (this.status === "shipped")
      throw new Error("Cannot cancel a shipped order.");
    this.status = "cancelled";
  }
}

// -- DTO -- flat, no logic; used in API layer ------------------
interface OrderDTO {
  orderId:      string;
  customerName: string;
  totalAmount:  number;
  status:       string;
}

// Mapper function -- converts domain -> DTO
function toOrderDTO(order: Order, id: string, customer: string): OrderDTO {
  return {
    orderId:      id,
    customerName: customer,
    totalAmount:  order.totalPrice(),
    status:       "pending",
  };
}
```

2.3.3 Observability

Definition

Observability is the ability to understand internal system state from external outputs. The three pillars are: **Logs**, **Metrics**, and **Traces**.



3 Unified Modelling Language (UML)

Definition

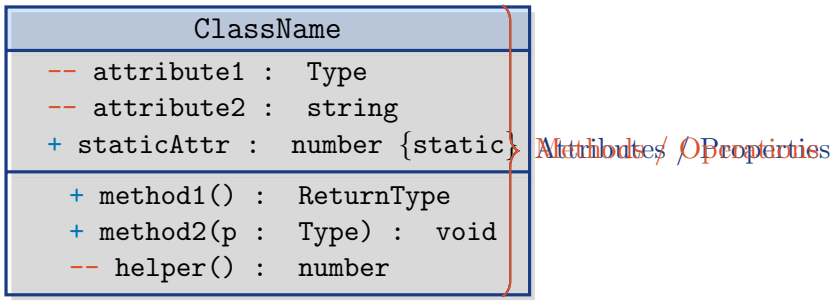
UML is a standardised visual language for describing, visualising, and documenting software system artefacts, maintained by the Object Management Group (OMG).

3.1 Structural UML Diagrams

3.1.1 Class Diagram

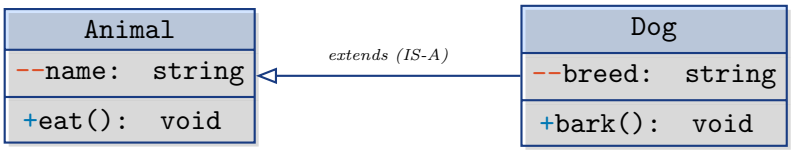
Definition

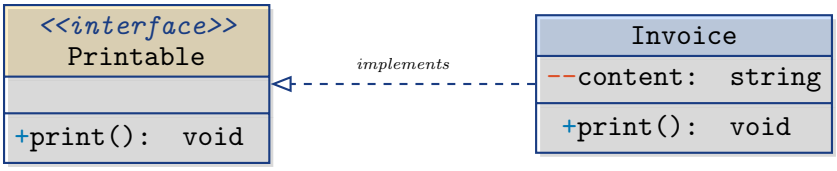
A **Class Diagram** shows classes, their attributes & operations, and the relationships between them. It is the most commonly used UML diagram.

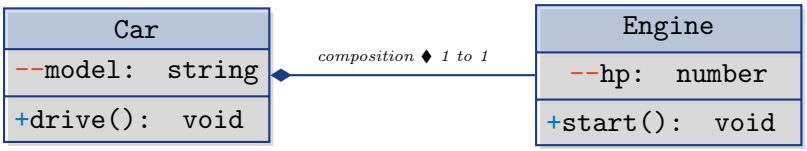


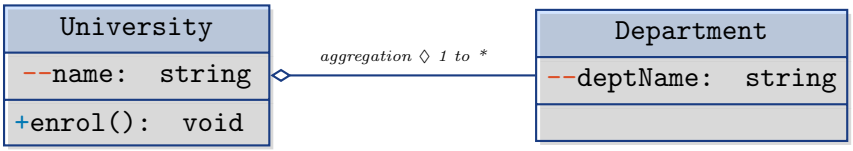
Visibility: + Public − Private # Protected ~ Package

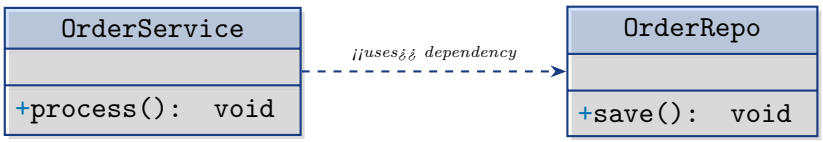
3.1.2 Class Relationships











UML Class Relationship Summary

Relationship	Description	Notation
Association	General link	Solid line
Directed Assoc.	One uses the other	Solid + open arrow
Aggregation	Weak HAS-A	Solid + open diamond ◊
Composition	Strong HAS-A	Solid + filled diamond ♦
Inheritance	IS-A (extends)	Solid + hollow triangle
Realisation	Implements interface	Dashed + hollow triangle
Dependency	Transient use	Dashed + open arrow

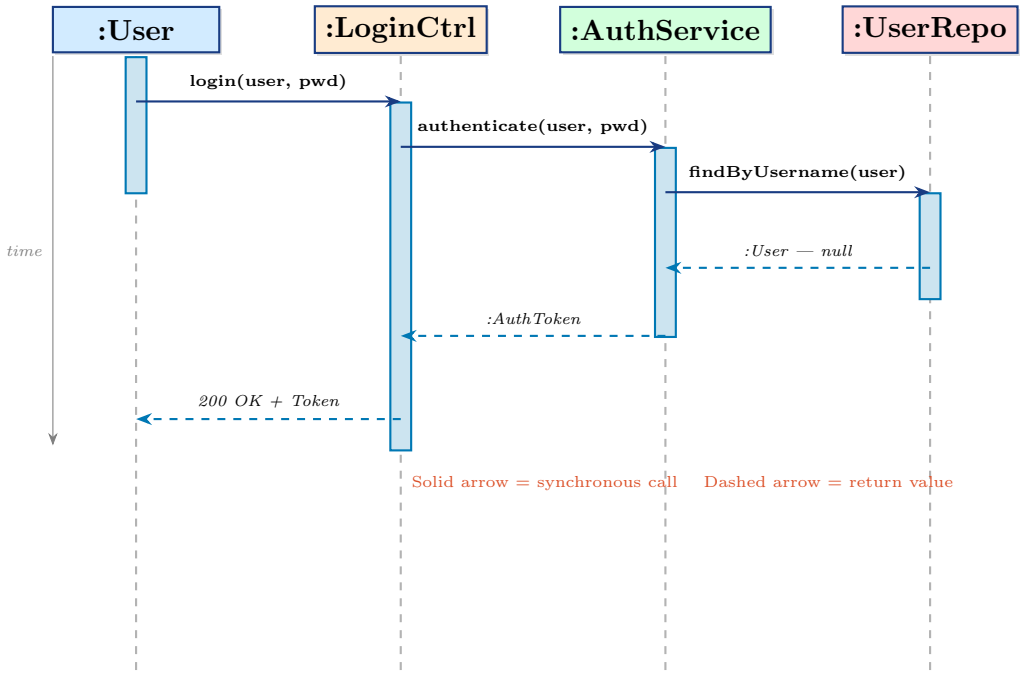
Multiplicity notation: `1` exactly one `0..1` zero or one `*` zero or more `1..*` one or more `m..n` from m to n.

3.2 Behavioural UML Diagrams

3.2.1 Sequence Diagram

Definition

A **Sequence Diagram** shows how objects interact in time-ordered sequence. It depicts message passing between participants along a vertical time axis.



Key Elements:

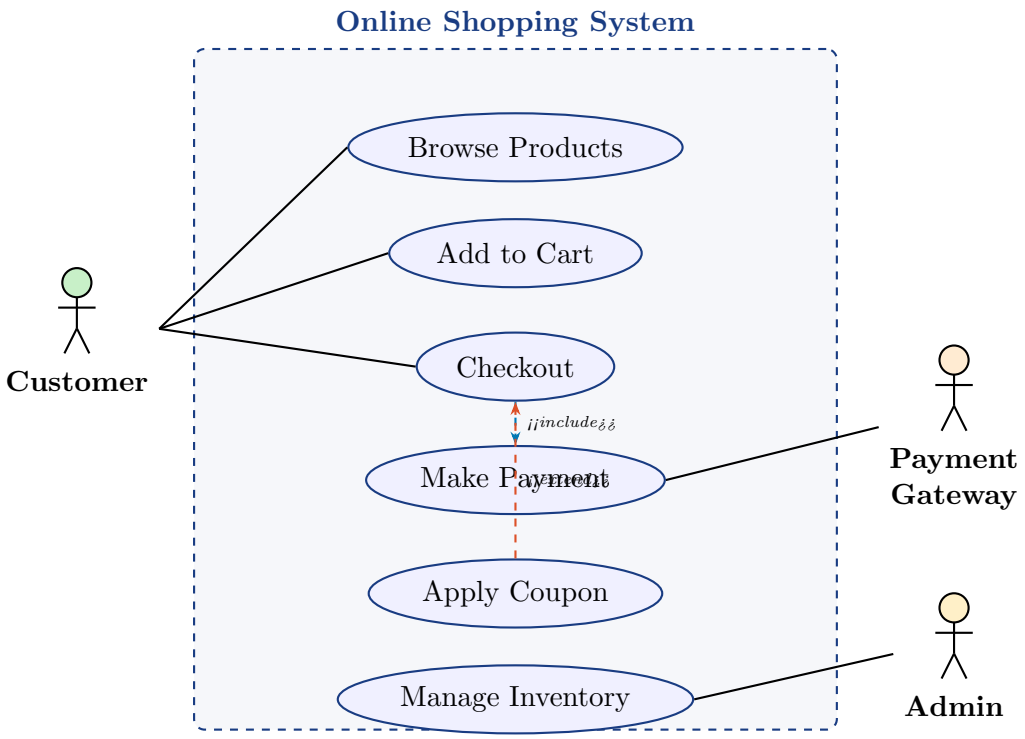
- **Lifeline**: vertical dashed line + box header = one participant.
- **Activation bar**: narrow rectangle showing active processing period.
- **Synchronous message**: solid arrowhead (caller blocks until return).

- **Return**: dashed arrow with open arrowhead.
- **Combined Fragments**: `alt` (if/else), `loop`, `opt` (optional), `par` (parallel).

3.2.2 Use Case Diagram

Definition

A **Use Case Diagram** captures *what* the system does and *who* interacts with it from the user's perspective, without detailing the how.



Key Elements:

- **Actor**: external entity (person/system) — stick figure.
- **Use Case**: system function — oval.
- **System Boundary**: rectangle enclosing all use cases.
- **¡¡include¿¿**: mandatory inclusion (Checkout always includes Make Payment).
- **¡¡extend¿¿**: conditional/optional behaviour (Apply Coupon extends Checkout).

4 Entity Relationship Diagram (ERD)

Definition

An **ERD** is a graphical model representing entities, attributes, and relationships in a database. It is the primary tool for conceptual database design.

4.1 Components

4.1.1 Entities

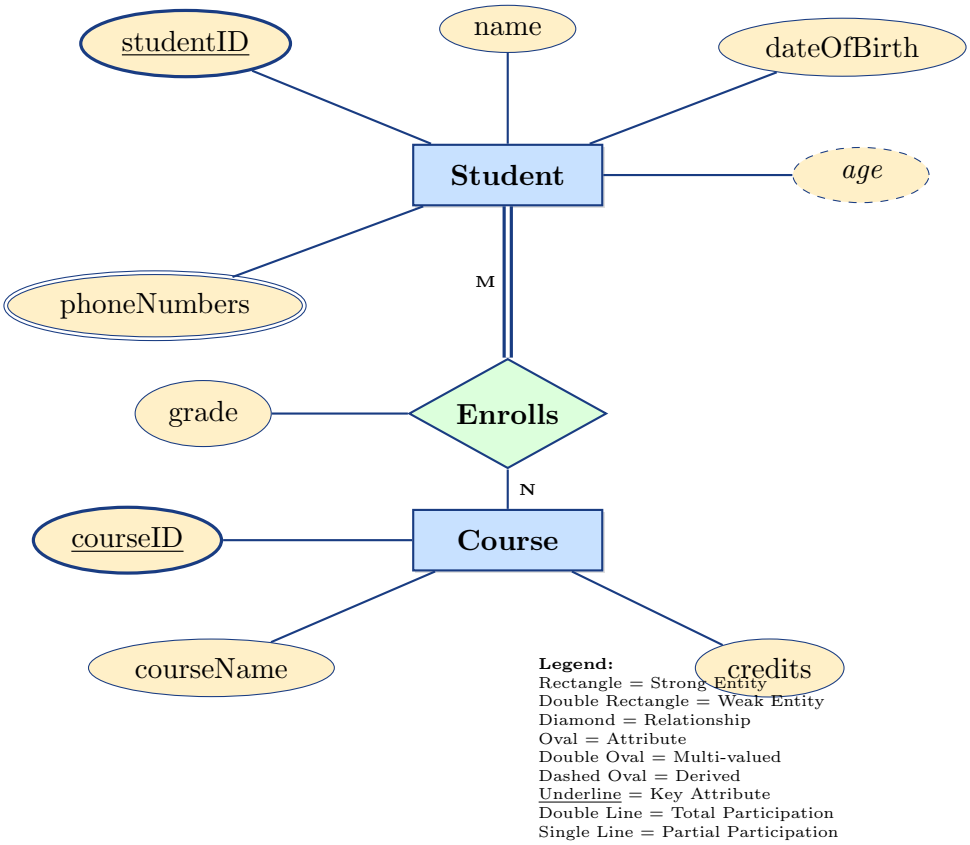
- **Strong Entity**: exists independently; has its own primary key. Drawn as a *rectangle*.
- **Weak Entity**: cannot exist without a related strong entity; identified by a partial key + parent key. Drawn as a *double rectangle*.

4.1.2 Attributes

Type	Description	Chen Symbol
Simple	Atomic; cannot be subdivided	Single oval
Composite	Can be divided (e.g. fullName → first + last)	Oval with children
Single-valued	Holds one value	Single oval
Multi-valued	Multiple values (e.g. phoneNumbers)	Double oval
Derived	Computed from others (e.g. age from DOB)	Dashed oval
Key	Uniquely identifies entity	Oval with underlined text

4.2 ERD Notations

4.2.1 Chen Notation



4.2.2 Crow's Foot Notation

Crow's Foot Symbol Reference

| | Both bars = exactly one (mandatory)
○ | Circle + bar = zero or one (optional)
< | Crow + bar = one or many (mandatory many)
○ < Circle + crow = zero or many (optional many)

4.3 Mapping Cardinalities

Cardinality Types

One-to-One (1:1)
Each A instance ↔ at most one B instance.
Example: Person ↔ Passport.

One-to-Many (1:N)
One A instance → many B instances; each B → one A.
Example: Department has many Employees.

Many-to-One (N:1)
Many A instances → one B instance. (Reverse of 1:N.)
Example: Many Orders placed by one Customer.

Many-to-Many (M:N)
Many A ↔ many B. Requires a *junction table*.
Example: Student enrolls in many Courses; Course has many Students.

4.4 Participation Constraints

Total Participation
Every entity instance *must* participate. *Mandatory*. Double line in Chen; mandatory symbol in Crow's Foot.
Example: Every Employee *must* work for a Department.

Partial Participation
Some instances may not participate. *Optional*. Single line in Chen; circle symbol in Crow's Foot.
Example: Not every Employee manages a Department.

5 Design Patterns

Definition

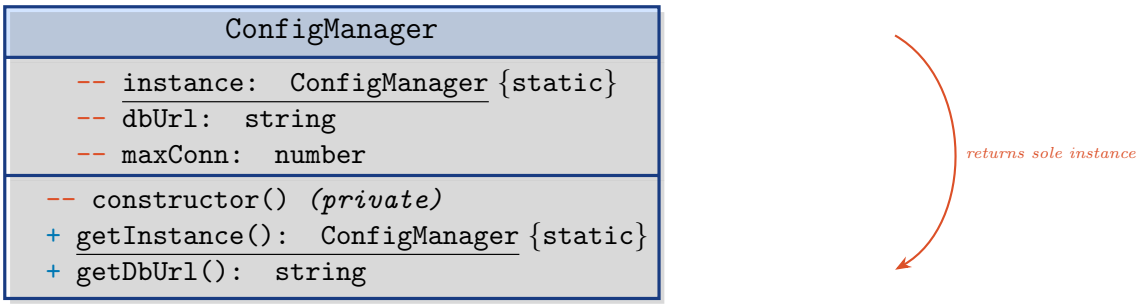
Design Patterns are reusable, general solutions to commonly occurring problems in software design. They are templates, not finished code.

Category	Purpose	Examples
Creational	Object creation mechanisms	Singleton, Factory, Builder
Structural	Composition of classes/objects	Adapter, Decorator, Composite
Behavioural	Communication between objects	Observer, Strategy, Command

5.1 Singleton Design Pattern

Definition

The **Singleton Pattern** ensures a class has **only one instance** and provides a *global point of access* to it. It is a *Creational* pattern.



Underlined = static member *Private constructor prevents* `new`

Listing 11: Singleton – Eager Initialisation (TypeScript)

```typescript
class ConfigManager {
  // Single instance created when class is first loaded
  private static readonly INSTANCE = new ConfigManager();

  private dbUrl: string;
  private maxConnections: number;

  // Private constructor prevents external instantiation
  private constructor() {
    this.dbUrl           = "postgresql://localhost:5432/mydb";
    this.maxConnections = 10;
  }

  static getInstance(): ConfigManager {
    return ConfigManager.INSTANCE;
  }

  getDbUrl(): string          { return this.dbUrl; }
```

```typescript
  getMaxConnections(): number { return this.maxConnections; }
}

// Usage -- always the same object
const cfg1 = ConfigManager.getInstance();
const cfg2 = ConfigManager.getInstance();
console.log(cfg1 === cfg2);          // true
console.log(cfg1.getDbUrl());
```

Listing 12: Singleton – Lazy Initialisation (TypeScript)

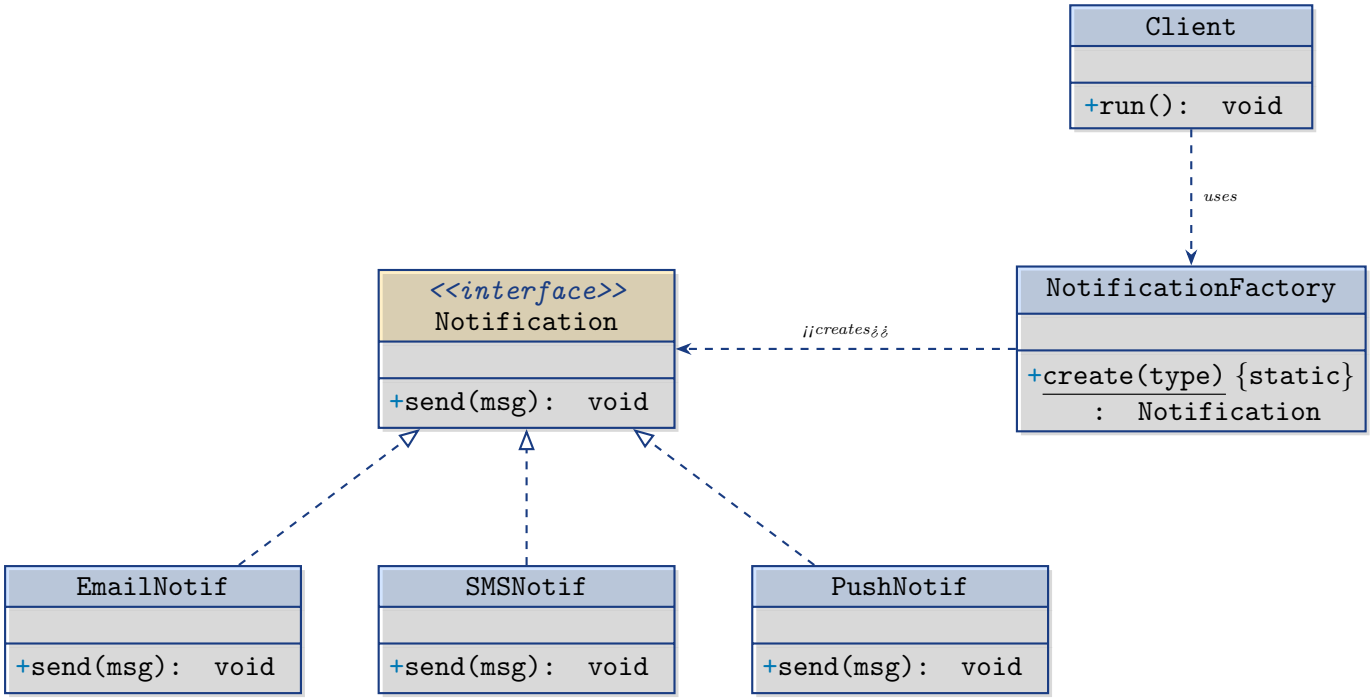
```typescript
class Logger {
  private static instance: Logger | null = null;
  private logs: string[] = [];

  private constructor() {}   // prevent external `new`

  static getInstance(): Logger {
    if (!Logger.instance) {
      Logger.instance = new Logger();
    }
    return Logger.instance;
  }

  log(message: string): void {
    const entry = `[${new Date().toISOString()}] ${message}`;
    this.logs.push(entry);
    console.log(entry);
  }

  getLogs(): string[] { return [...this.logs]; }
}

// Always same Logger instance across application
Logger.getInstance().log("Application started");
Logger.getInstance().log("User logged in");
console.log(Logger.getInstance().getLogs().length); // 2
```

	Description
Advantages	
Single instance	Exactly one object guaranteed.
Global access	Accessible from anywhere via `getInstance()`.
Lazy init	Can defer creation to first use.
Disadvantages	
Global state	Hidden global state makes code hard to test.
Tight coupling	Callers are coupled to the concrete Singleton class.
Testing	Difficult to mock or substitute in unit tests.

5.2 Factory Design Pattern

Definition

The **Factory Pattern** provides an interface for **creating objects** but lets subclasses or configuration **decide which class to instantiate**. Replaces direct `new` calls with a factory method.



Listing 13: Simple Factory in TypeScript

```
// Product interface (contract)
interface Notification {
  send(message: string): void;
}

// Concrete products
class EmailNotification implements Notification {
  send(message: string): void {
    console.log(`[EMAIL] ${message}`);
  }
}

class SMSNotification implements Notification {
  send(message: string): void {
    console.log(`[SMS] ${message}`);
  }
}

class PushNotification implements Notification {
  send(message: string): void {
    console.log(`[PUSH] ${message}`);
```

```
  }
}

// Simple Factory -- centralises creation
type NotifType = "email" | "sms" | "push";

class NotificationFactory {
  static create(type: NotifType): Notification {
    switch (type) {
      case "email": return new EmailNotification();
      case "sms":   return new SMSNotification();
      case "push":  return new PushNotification();
    }
  }
}

// Client -- decoupled from concrete classes
const notif: Notification = NotificationFactory.create("email");
notif.send("Your order has shipped!");   // [EMAIL] Your order
    has shipped!
```

Listing 14: Factory Method Pattern (GoF) in TypeScript

```
// Abstract Product
interface Button {
  render(): void;
  onClick(): void;
}

// Concrete Products
class WindowsButton implements Button {
  render(): void  { console.log("Rendering Windows button"); }
  onClick(): void { console.log("Windows button clicked"); }
}

class HTMLButton implements Button {
  render(): void  { console.log("Rendering HTML button"); }
  onClick(): void { console.log("HTML button clicked"); }
}

// Abstract Creator
abstract class Dialog {
  // Factory Method -- subclasses decide what to create
  abstract createButton(): Button;

  // Template method using factory method
  render(): void {
    const btn = this.createButton();
    btn.render();
    btn.onClick();
  }
}
```

```

// Concrete Creators
class WindowsDialog extends Dialog {
  createButton(): Button { return new WindowsButton(); }
}

class WebDialog extends Dialog {
  createButton(): Button { return new HTMLButton(); }
}

// Client
function runApp(os: "windows" | "web"): void {
  const dialog: Dialog = os === "windows"
    ? new WindowsDialog()
    : new WebDialog();
  dialog.render();
}

runApp("web");       // Rendering HTML button  /  HTML button clicked
runApp("windows");   // Rendering Windows button / Windows button clicked
```

Listing 15: Abstract Factory Pattern in TypeScript

```
// Abstract products
interface GUIButton   { render(): void; }
interface GUICheckbox { render(): void; }

// Concrete products
class WinButton   implements GUIButton   { render() { console.log("Win Button");   } }
class WinCheckbox implements GUICheckbox { render() { console.log("Win Checkbox"); } }
class MacButton   implements GUIButton   { render() { console.log("Mac Button");   } }
class MacCheckbox implements GUICheckbox { render() { console.log("Mac Checkbox"); } }

// Abstract Factory
interface GUIFactory {
  createButton():   GUIButton;
  createCheckbox(): GUICheckbox;
}

// Concrete Factories
class WindowsFactory implements GUIFactory {
  createButton():   GUIButton   { return new WinButton();   }
  createCheckbox(): GUICheckbox { return new WinCheckbox(); }
}

class MacFactory implements GUIFactory {
```

```typescript
  createButton():   GUIButton   { return new MacButton();   }
  createCheckbox(): GUICheckbox { return new MacCheckbox(); }
}

// Client -- only depends on abstract interfaces
class Application {
  private btn: GUIButton;
  private chk: GUICheckbox;

  constructor(factory: GUIFactory) {
    this.btn = factory.createButton();
    this.chk = factory.createCheckbox();
  }

  render(): void { this.btn.render(); this.chk.render(); }
}

const isWindows = true;
const factory: GUIFactory = isWindows ? new WindowsFactory() :
   new MacFactory();
new Application(factory).render();
```

	Description
Advantages	
Decoupling	Client decoupled from concrete product classes.
Open/Closed	Add new products without changing client code.
Single Responsibility	Creation logic isolated in the factory.
Disadvantages	
Complexity	Introduces extra classes and abstractions.
Indirection	Harder to trace which object is actually created.

Quick Revision Summary

SDSE Mid-Semester – Key Terms at a Glance

Term	One-Line Definition
SDLC	Structured process to plan, build, test, and maintain software.
Quality Attribute	Non-functional requirement (performance, security, reliability...).
Class	Blueprint defining structure and behaviour of objects.
Object	Concrete instance of a class with its own state.
Constructor	Initialises an object to a valid state upon creation.
Invariant	Condition always true for every instance of a class.
Encapsulation	Bundle data + methods; restrict direct field access.
Getter/Setter	Methods to read/write private fields with optional validation.
Abstraction	Hide implementation; expose only essential features.
Abstract Class	Partial template; cannot be instantiated; may have abstract methods.
Interface	Contract of method signatures a class must implement.
Inheritance	Derive attributes/methods from a parent class (IS-A).
Polymorphism	One interface, many forms; correct method resolved at runtime.
Composition	Strong HAS-A; component lifetime tied to owner.
Aggregation	Weak HAS-A; component exists independently.
DTO	Plain data carrier between layers; no business logic.
Observability	Understand system state via Logs, Metrics, and Traces.
Class Diagram	Static UML diagram showing classes and relationships.
Sequence Diagram	Behavioural UML showing time-ordered message passing.
Use Case Diagram	Shows actors and system functions (what, not how).
ERD	Conceptual data model: entities, attributes, relationships.
Cardinality	Number of entity instances in a relationship (1:1, 1:N, M:N).
Total Participation	Every entity instance must participate (mandatory).
Partial Participation	Some entity instances may not participate (optional).
Chen Notation	Academic ERD using rectangles, diamonds, ovals.
Crow's Foot	Compact ERD using line-end symbols for cardinality.
Singleton	Ensures a class has exactly one global instance.
Factory	Centralises object creation; decouples client from concrete types.